

PRELIMINARY TECHNICAL REPORT

Development of a Maintenance Risk and Resource Allocation Component

for a Facility Decision Support Framework

Version	0.1
Date	August 22, 2026
Status	Preliminary research and development report
Author	Oluwafemi Olaiyapo
Repository	github.com/oluwafemi244/facility-maintenance-energy-decision-support

Scope note

This report covers the maintenance component only. The facility energy system component of the framework has not been started and is described here as planned work, not completed work.

Contents

Executive Summary

1. Purpose and Scope
2. Development Approach
3. Source Data
4. Failure Risk Modeling
5. Maintenance Consequence and Priority
6. Resource Allocation Demonstration
7. Sensitivity Analysis
8. Current Limitations
9. Reproducibility
10. Current Development Status
11. Next Development Stages
12. Conclusion

References

Executive Summary

This report covers the first working piece of a larger framework I am building for facility maintenance and energy decision support. The framework is meant to connect what is known about equipment condition to a defensible answer to a question every maintenance organization faces: when the available budget and crew hours will not cover everything that needs doing, which work gets done. Version 0.1 addresses the maintenance side of that question. The energy side has not been started.

The proof of concept is built on the AI4I 2020 Predictive Maintenance Dataset from the UCI Machine Learning Repository, a synthetic set of 10,000 observations. I chose it for its structure rather than its realism. It pairs equipment operating variables with a failure outcome, which makes it suitable for testing the initial failure risk modeling process.

The work completed covers a data quality review, exploratory analysis, a comparison of four failure risk models, estimated failure probabilities, a maintenance risk score that combines those probabilities with consequence, a priority ranking, a constrained allocation of maintenance work, and sensitivity testing of the whole sequence.

One property of the dataset shaped every modeling decision that followed. Only 339 of the 10,000 observations are labeled as failures, a rate of 3.39 percent. At that prevalence a model that predicts no failure at all is right 96.6 percent of the time, so ordinary accuracy is not a usable measure and is not reported as one anywhere in this report.

Four classifiers were evaluated: a majority class dummy model, logistic regression, a decision tree, and a random forest. The random forest produced the strongest ROC AUC, average precision, precision, and F1

score on held out test data. The decision tree caught substantially more of the actual failures. That is not a tie to be broken by picking a higher number, it is a real trade between finding failures and generating false alerts, and which side of it a facility should sit on is a maintenance decision rather than a modeling one.

For the prioritization demonstration, estimated failure probability was combined with simulated operational criticality and expected downtime, and the resulting scores were tested against simulated budget, labor, and spare parts limits. Under a demonstration budget of \$120,000 and 240 available labor hours, the optimization model selected 11 maintenance actions costing \$83,656 and consuming 236 labor hours, addressing a total modeled risk score of approximately 107.77.

Ranking by risk score and working down the list until the resources ran out selected 12 actions, spent \$95,572, and used all 240 hours to address the same modeled risk. I want to be careful about what that comparison shows. It does not establish that optimization beats ranking in general. It shows that a constrained allocation model can be implemented and put side by side with the simpler approach on the same portfolio, which is the point of the exercise at this stage.

Sensitivity analysis showed that the highest priority assets in this portfolio held their positions when the weighting between criticality and downtime was changed, and that adding budget or labor stops helping once the other constraint becomes the binding one.

None of this is validated for operational use. The equipment dataset is synthetic, the facility context variables are simulated, and no output from the current proof of concept has been operationally validated or used to support maintenance decisions at an actual facility. What exists is a reproducible computational process that has been implemented, tested, and documented well enough to be extended and evaluated against better data.

Key results at a glance

Item	Current result
Source data	10,000 synthetic observations
Machine failure prevalence	339 observations, 3.39 percent
Highest test recall	Decision tree, 0.8824
Highest test ROC AUC	Random forest, 0.9636
Optimization result	11 actions, \$83,656 cost, 236 labor hours
Modeled risk score addressed	107.77
Validation status	Internal proof of concept only

Table 1. Summary of the version 0.1 results.

1. Purpose and Scope

The broader project is a computational decision support framework for infrastructure and energy intensive industrial facilities. It has two workstreams. The first concerns equipment condition, maintenance prioritization, and the allocation of maintenance resources. The second will examine facility energy system performance and reliability under different operating and disruption scenarios. This report covers the first workstream only.

The framework is not intended to replace equipment inspection, manufacturer requirements, engineering judgment, established maintenance procedures, safety requirements, or the decisions of qualified personnel. It is intended to provide a transparent ordering of maintenance priorities that qualified personnel can review, challenge, and adjust using facility specific information.

The goal of version 0.1 is narrower still. It is to find out whether the analytical sequence I proposed can actually be implemented in a transparent and reproducible form, and to surface the technical problems that have to be solved before any of it can be validated.

1.1 The maintenance sequence as implemented



Figure 1. The analytical sequence implemented in version 0.1. Each stage produces an artifact the next stage consumes, and each is kept separately in the repository.

2. Development Approach

The work was staged so that each piece could be reviewed on its own, and each stage is retained separately in the repository along with its assumptions, code, outputs, and limitations.

The first stage set up the public repository, documented the planned framework, identified the source dataset, and built a data dictionary. The second examined the dataset for structure, missing values, duplicates, failure frequency, failure mode frequency, and inconsistencies between the general failure

indicator and the individual mode variables. The third developed the baseline risk models and compared their behavior across several measures rather than a single headline score.

The fourth stage connected estimated failure probability to the simulated consequence variables to produce a preliminary maintenance risk score. The fifth implemented the constrained allocation model using simulated cost, labor, and parts availability. The sixth tested how much of the result depended on the assumptions, by varying the consequence weights, the budget, the labor limit, and combinations of the two.

The order matters. Each stage was only useful because the one before it produced something concrete to feed it, and building them in sequence exposed problems, particularly in the labeling of the source data, that would have been easy to miss if the modeling had come first.

3. Source Data

3.1 AI4I 2020 Predictive Maintenance Dataset

Version 0.1 uses the AI4I 2020 Predictive Maintenance Dataset published through the UCI Machine Learning Repository. It contains 10,000 observations across 14 variables.

The dataset is synthetic. Its values were generated from a model of machine behavior rather than logged from equipment in service, and it is not presented here as data from an operating facility. It also describes an industrial machining process rather than building mechanical equipment. Both facts constrain what any result in this report can mean, and section 8 sets out the consequences.

Variable	Meaning	Role in this work
UDI	Row identifier	Excluded
Product ID	Serial identifier	Excluded
Type	Product quality class, L, M or H	Predictor
Air temperature	Ambient temperature, kelvin	Predictor
Process temperature	Process temperature, kelvin	Predictor
Rotational speed	Equipment speed, rpm	Predictor
Torque	Mechanical torque, newton metres	Predictor
Tool wear	Accumulated wear, minutes	Predictor
Machine failure	Observed failure	Target
Tool wear failure	Failure mode indicator	Excluded, component of target
Heat dissipation failure	Failure mode indicator	Excluded, component of target
Power failure	Failure mode indicator	Excluded, component of target

Variable	Meaning	Role in this work
Overstrain failure	Failure mode indicator	Excluded, component of target
Random failure	Failure mode indicator	Excluded, see section 3.2

Table 2. The source variables and how each is used.

3.2 Data quality review

The dataset contains no missing values and no duplicate rows. Failures occur in 339 of the 10,000 observations, a rate of 3.39 percent, and the individual failure modes break down as follows.

Failure mode	Count
Heat dissipation failure	115
Overstrain failure	98
Power failure	95
Tool wear failure	46
Random failure	19

Table 3. Failure mode counts. The modes sum to more than 339 because some observations carry more than one.

Two inconsistencies between the general failure indicator and the mode indicators are worth recording, because both affect how the target should be read.

Nine observations are labeled as failures with none of the five mode indicators set. There is no recorded explanation for them in the source documentation.

Separately, 18 observations carry a mode indicator while the general failure indicator remains zero, and every one of those 18 is a random failure flag. None of the four substantive modes ever appears on an observation labeled as no failure. In practical terms this means random failure is effectively absent from the target, and any analysis that treats it as a mode contributing to the label will be wrong.

I retained all of these records. The project does not quietly drop or alter source rows because the target variables do not line up perfectly, and with a positive class this small, discarding ambiguous positives would distort the evaluation more than the ambiguity does.

4. Failure Risk Modeling

4.1 Predictor selection

The models use six variables: product type as an equipment category, air temperature, process temperature, rotational speed and torque as operating variables, and tool wear as the condition related variable. Machine failure is the target.

The five mode indicators are excluded from the predictor set because they describe the failure event itself and would leak the target. This is not a hypothetical concern. The five mode indicators are excluded from the predictor set because they describe failure events and would introduce target leakage. Including variables that directly encode the outcome would make the resulting performance measures unsuitable for evaluating whether operating condition variables can predict failure.

4.2 Training and test data

The data was split 80 percent training and 20 percent test, stratified on the target so that the low failure prevalence carried through to both sides. That produced 8,000 training observations at a failure rate of approximately 3.39 percent and 2,000 test observations at approximately 3.40 percent.

4.3 Models evaluated

Four models were evaluated: a majority class dummy classifier, logistic regression, a decision tree, and a random forest.

The dummy classifier earns its place in that list. When failures are this uncommon, a model that predicts no failure for every observation still posts a high ordinary accuracy, and having that number in the results table makes the point better than any amount of explanation.

4.4 Test set results

Model	Accuracy	Balanced acc.	Precision	Recall	F1	ROC AUC	Avg. prec.
Dummy baseline	0.9660	0.5000	0.0000	0.0000	0.0000	0.5000	0.0340
Logistic regression	0.8245	0.8240	0.1418	0.8235	0.2419	0.9070	0.3818
Decision tree	0.9290	0.9065	0.3093	0.8824	0.4580	0.9111	0.5901
Random forest	0.9800	0.7272	0.9118	0.4559	0.6078	0.9636	0.7762

Table 4. Held out test set performance. Metrics at the default classification threshold.

The dummy classifier reached 96.60 percent accuracy while identifying not a single failure, which is the clearest available demonstration of why accuracy is not the measure to optimize here.

Logistic regression found roughly 82 percent of the actual failures but at low precision, so most of what it flagged was not a failure. The decision tree found the most failures of any substantive model, approximately 88 percent, at a precision of about 31 percent, meaning roughly two in three of its alerts were false. The random forest inverted that: precision of approximately 91 percent, and recall at the default threshold of approximately 46 percent, so it missed more than half the failures but was usually right when it spoke.

No model is best on every measure, and at this stage there is no basis for calling one superior. Which trade a facility should accept depends on what a missed failure costs relative to an unnecessary inspection, and that is not a question this dataset can answer.

4.5 Cross validation

Five fold stratified cross validation on the training data produced the same picture, which is the main thing I wanted from it.

Model	Balanced acc.	Precision	Recall	F1	ROC AUC	Avg. prec.
Logistic regression	0.8138	0.1374	0.8044	0.2346	0.8965	0.4409
Decision tree	0.9165	0.2990	0.9078	0.4495	0.9180	0.5694
Random forest	0.6912	0.8973	0.3839	0.5366	0.9669	0.7546

Table 5. Five fold stratified cross validation, training data, mean across folds.

The decision tree again returned higher recall and the random forest again returned higher precision and stronger ranking measures. The cross validation results show that the same general tradeoff appears across multiple training folds rather than only in the held out test split.

4.6 Random forest variable importance

Variable	Importance
Rotational speed	0.3022
Torque	0.3002
Tool wear	0.1995
Air temperature	0.1040
Process temperature	0.0746

Table 6. Random forest variable importance. The remainder is spread across the encoded product type categories.

Rotational speed and torque carry the most weight, followed by tool wear. These values describe how the fitted forest behaves. They are not evidence of a causal relationship between any variable and equipment failure, and they should not be read as one.

4.7 Probability model used downstream

The random forest probabilities feed the prioritization demonstration, on the strength of its ROC AUC and average precision in these experiments. That is a working choice for version 0.1 and not a selection of an operational model. Probability calibration, threshold analysis, additional datasets, and validation all remain to be done, and calibration matters more than it might appear: the priority score multiplies probability by a consequence term, so a systematically inflated or deflated probability propagates straight into the ranking.

5. Maintenance Consequence and Priority

5.1 Why probability alone is not enough

Failure probability by itself does not determine maintenance priority. Two assets with the same probability of failing can matter to a facility in completely different ways, and a lower probability failure often deserves to go first because the equipment serves a more important function or because its failure would take longer to recover from. This is the reason the framework keeps likelihood and consequence separate rather than learning a single combined score. Fusing them is simpler to build and impossible to audit, because a change in the ranking can no longer be attributed to a change in equipment condition or a change in organizational priorities.

5.2 Simulated facility context

The AI4I dataset contains no operational criticality, expected downtime, maintenance cost, labor requirement, spare parts availability, or budget. These are the exact inputs the consequence layer needs, and none of them can be recovered from the source data.

Statement on simulated values

The facility context variables used in sections 5, 6 and 7 were created separately as simulated demonstration values. They are not measurements or records from any operating facility, and no result computed from them is an estimate of anything. A fixed random seed is used so they reproduce exactly.

They are kept in a separate file, clearly identified, and never merged into the public dataset or presented as part of it.

5.3 Consequence index

The preliminary consequence index combines two factors, operational criticality and expected downtime, both normalized before they are combined. The demonstration assigns 60 percent of the weight to criticality and 40 percent to downtime.

Those weights are assumptions made to demonstrate the calculation. They are not industry standards and they have not been validated for any facility. I have exposed them rather than burying them in the code, because the relative value an organization places on continuity of service against duration of

outage is a management decision and not a modeling one. Section 7.1 reports how much the ranking actually depends on them.

5.4 Maintenance risk score

$$\text{Maintenance risk} = \text{estimated failure probability} \times \text{consequence index}$$

The result is rescaled from 0 to 100 for presentation. The priority categories used in the demonstration are illustrative labels for reading the output, not maintenance standards and not operational action thresholds.

6. Resource Allocation Demonstration

6.1 Demonstration constraints

Resource	Demonstration limit
Maintenance budget	\$120,000
Maintenance labor	240 hours
Spare parts	Action permitted only where parts are available

Table 7. Simulated resource constraints for the allocation demonstration.

The point of imposing these limits is to force the situation that makes prioritization necessary in the first place, where the candidate work exceeds what can be done and something has to be deferred.

6.2 Simple ranking

The first method sorts candidate actions by maintenance risk score and works down the list, selecting each action while budget and labor remain. It selected 12 actions, spending \$95,572 and all 240 available labor hours, and addressed a total modeled risk score of approximately 107.77.

6.3 Constrained optimization

The second method is a binary optimization model that maximizes the total maintenance risk score of the selected actions subject to the budget, the available labor hours, and parts availability. The solver reached an optimal status. It selected 11 actions, spending \$83,656 and 236 labor hours, and addressed the same total modeled risk score of approximately 107.77.

6.4 Comparison

Method	Actions	Total cost	Labor used	Risk addressed
Simple risk ranking	12	\$95,572	240	107.77

Method	Actions	Total cost	Labor used	Risk addressed
Constrained optimization	11	\$83,656	236	107.77

Table 8. Allocation results under identical constraints.

Both approaches addressed the same modeled risk, and the optimization happened to do it with fewer actions, less money, and slightly less labor. That difference is worth less than it looks. The objective only maximizes risk addressed; it does not minimize cost once that maximum is reached, so the cheaper solution here is a property of this particular portfolio rather than a general result. Nothing in this comparison establishes that optimization is more efficient than ranking.

What the demonstration does establish is that maintenance risk estimates can be carried through into an explicit resource decision, and that the two approaches can be run against the same portfolio and compared on the same terms. That was the objective for this stage.

7. Sensitivity Analysis

Every number in sections 5 and 6 rests on assumptions I chose. Sensitivity analysis is how I find out which of those assumptions the results actually depend on.

7.1 Consequence weight sensitivity

Five weighting combinations were tested across the range from criticality dominant to downtime dominant.

Criticality weight	Downtime weight
0.20	0.80
0.40	0.60
0.50	0.50
0.60	0.40
0.80	0.20

Table 9. Consequence weighting combinations tested.

Nine assets stayed inside the top ten under all five combinations, and the largest rank movement observed anywhere was four positions. For this portfolio the top of the ranking is reasonably stable against the weighting choice, which is the desirable outcome: a ranking that reordered completely under a small weight change would be reporting my assumptions rather than the data.

This applies to this demonstration portfolio only. It does not establish that the method will be equally stable on other datasets or at a real facility.

7.2 Budget sensitivity

Holding labor fixed at 240 hours and varying the budget:

Budget	Actions selected	Risk addressed
\$40,000	6	104.09
\$60,000	8	106.51
\$80,000	11	107.56
\$100,000	11	107.77
\$120,000	11	107.77
\$150,000	11	107.77
\$180,000	11	107.77

Table 10. Budget sensitivity at fixed labor of 240 hours.

More budget improves the outcome at the lower levels, then stops. Past \$100,000 the result does not move, because budget is no longer what is holding the program back.

7.3 Labor sensitivity

Holding the budget fixed at \$120,000 and varying available labor:

Available labor	Actions selected	Risk addressed
80 hours	3	96.65
120 hours	6	104.09
160 hours	8	105.86
200 hours	9	106.82
240 hours	11	107.77
300 hours	14	108.60
360 hours	15	108.74

Table 11. Labor sensitivity at fixed budget of \$120,000.

Labor keeps paying off over a wider range than budget does, though the gain between 300 and 360 hours is small enough to suggest the portfolio is running out of work worth doing rather than running out of hours.

7.4 Combined resource conditions

Varying budget and labor together shows the limiting resource moving between them. At 120 available hours, adding budget changed nothing, because the crew could not complete more work regardless of what it cost. At a \$60,000 budget, adding labor eventually stopped helping for the mirror image reason.

This is the practical argument for treating maintenance planning as a joint allocation problem. Evaluating equipment risk, budget, and labor separately will produce a plan that looks affordable and cannot be staffed, or staffable and cannot be paid for.

7.5 Portfolio composition

Priority category	Assets	Percentage
Low	57	95.00%
Moderate	1	1.67%
High	2	3.33%

Table 12. Preliminary priority composition of the 60 asset demonstration portfolio.

No asset falls into the illustrative Critical category at all, and I want to flag this as a genuine weakness rather than let it pass. The portfolio is heavily concentrated in low modeled risk, which means the demonstration proves the process executes but exercises only a narrow part of the decision surface it is supposed to handle. It should not be read as representative of the risk distribution at a real facility, and building a portfolio that spans the range properly is on the list for the next stage.

8. Current Limitations

The constraints below are stated so that no reader has to infer them.

- The equipment dataset is synthetic. Performance on it does not establish performance on real industrial equipment, where data quality, missing information, inconsistent maintenance records, and operating conditions may differ substantially.
- The facility context variables are simulated. Operational criticality, downtime, maintenance cost, labor requirements, spare parts availability, and the resource limits themselves were not obtained from an operating facility.
- The consequence weights and priority thresholds are demonstration assumptions rather than validated values.
- The models use a narrow set of operating variables. Real maintenance decisions also turn on inspection results, equipment age, maintenance history, vibration and lubrication condition, redundancy, safety consequences, production requirements, and equipment specific engineering limits, none of which are represented here.

- No model has been established as the failure risk model. The four tested show different strengths, and choosing between them requires information about the cost of a missed failure that this work does not have.
- The optimization model ignores technician skill categories, scheduling across time, maintenance sequencing, equipment dependencies, shutdown windows, production losses, uncertainty in cost and labor estimates, and interactions among simultaneous equipment outages. Any of these can override a technically correct ordering in practice.
- The demonstration portfolio is concentrated in low priority observations, as section 7.5 sets out.
- The facility energy system component has not been implemented.

Taken together these mean the correct reading of this report is that a process has been built and shown to run end to end, not that a system has been shown to work.

9. Reproducibility

The project is maintained in a public repository so that the development sequence can be followed and the main calculations reproduced. It holds the source data documentation and retrieval steps, a data dictionary, the exploratory analysis code and its saved outputs, the failure risk modeling code, the model performance results, the failure probability outputs, the simulated facility context stored separately from the source data, the maintenance priority results, the allocation results, the sensitivity analysis results, and the framework architecture, validation strategy, and limitations documentation.

Fixed random states are used throughout, and the boundary between source data and simulated project inputs is marked in the data files themselves rather than only in the documentation, so that no file can be separated from the statement of what it is.

10. Current Development Status

The table below states what has been implemented and what has not, without rounding either direction.

Component	Status	Evidence
Data documentation and dictionary	Implemented	Section 3
Exploratory and data quality analysis	Implemented	Section 3.2
Baseline failure risk models	Implemented	Section 4
Failure probability output	Implemented	Section 4.7
Consequence and maintenance risk score	Implemented, simulated inputs	Section 5
Maintenance priority ranking	Implemented, simulated inputs	Section 6.2

Component	Status	Evidence
Constrained resource allocation	Implemented, simulated inputs	Section 6.3
Sensitivity analysis	Implemented	Section 7
Probability calibration	Not started	Section 4.7
Operational validation	Not started	Section 8
Facility energy system component	Not started	Section 11

Table 13. Implementation status of the framework components.

This is not a completed operational system and the repository does not claim otherwise. What it shows is that development of the proposed framework has begun and that several of its core analytical components exist, run, and have been tested against their own assumptions.

11. Next Development Stages

The next round of maintenance work is about validation and refinement rather than new capability. That means testing additional public equipment datasets where suitable data exists, examining probability calibration, evaluating classification thresholds against an explicit cost of missed failures, building demonstration portfolios that span the priority range instead of clustering at the bottom, and extending the allocation formulation.

The optimization model in particular has obvious room to grow. Scheduling across time, maintenance windows, technician availability, and equipment dependencies are all constraints a real facility works under and none of them are in the current formulation.

A later stage begins the energy workstream, covering facility energy demand, supply resources, backup generation, storage, equipment availability, and disruption scenarios. The longer term objective is to connect maintenance condition and timing to facility energy continuity inside one documented framework, since the equipment whose failure matters most for energy continuity is frequently not the equipment a maintenance backlog would surface on its own.

12. Conclusion

This report documents the first implemented maintenance component of the planned facility maintenance and energy decision support framework. The work shows that equipment operating data can be carried through a reproducible sequence that estimates failure risk, adds operational consequence, produces a maintenance priority, and evaluates candidate actions under explicit resource constraints.

It also shows why those pieces cannot be considered separately. A model chosen on accuracy alone would have been the one that finds no failures. A priority order built on probability alone would have

ignored what a failure costs. The combined sensitivity analysis also showed that considering budget without labor, or labor without budget, can miss which resource is actually limiting the modeled allocation.

The results should be read inside their limits. The source dataset is synthetic, the facility context is simulated, and nothing here has been validated for operational use. The value of this stage is that the project has moved past a written concept: a working computational process, documented assumptions, saved outputs, sensitivity tests, and a validation plan now exist, and they give the next stage something concrete to build on and to argue with.

References

AI4I 2020 Predictive Maintenance Dataset. (2020). UCI Machine Learning Repository. DOI: 10.24432/C5HS5C.

UCI Machine Learning Repository. AI4I 2020 Predictive Maintenance Dataset.
<https://archive.ics.uci.edu/dataset/601/ai4i>

Repository

github.com/oluwafemi244/facility-maintenance-energy-decision-support